

Documentation des modules du DRaC

Pour garantir une architecture logicielle maintenable, évolutive et conforme aux standards de développement des Security Operations Centers (SOC), le code source du DRaC a été divisé en plusieurs scripts Python. Cette approche permet de séparer la logique de détection, d'orchestration et de remédiation.

Cette documentation technique explique le rôle et le fonctionnement de chaque composant de notre solution :

1. Les Outils

- **detector.py**

Ce module regroupe les fonctions de diagnostic actif. Il ne prend aucune décision, son rôle est purement informatif. Il utilise des librairies natives (**subprocess**, **socket**, **requests**) pour vérifier trois paramètres vitaux de la cible :

- **Disponibilité réseau** (Envoi de requêtes ICMP (Ping));
- **Disponibilité applicative** (Vérification de l'ouverture du port TCP/80 (Nginx));
- **Intégrité des données** (Téléchargement du code source de la page Web et calcul de son empreinte cryptographique (Hash SHA-256) pour la comparer avec l'empreinte saine stockée dans le coffre-fort (.env).

- **reactor.py**

C'est le module le plus sensible de l'architecture. Il contient toutes les fonctions interagissant directement avec l'hyperviseur via la bibliothèque **proxmoxer**. Il s'authentifie auprès de Proxmox grâce à un **Token d'API sécurisé**. Ses fonctions (Playbooks) incluent :

- **isoler_reseau_vm**

Elle déconnecte virtuellement la carte réseau (link_down=1);

- **bloquer_ip_via_proxmox**

Elle injecte une règle de pare-feu (DROP) au niveau du nœud (Datacenter);

- **restaurer_snapshot**

Elle orchestre la chaîne :

Isolement de la VM - Création d'un Snapshot pour garder les preuves - Rollback vers l'état de référence sain - Redémarrage de la machine.

- **notifier.py**

Ce module établit la communication avec l'administrateur. Il utilise la bibliothèque **smtplib** pour établir un tunnel chiffré (SSL/TLS) avec un serveur de messagerie. Il contient des modèles pré-rédigés pour chaque type d'incident (Alerte Défaçage, Confinement, Résolution), assurant un reporting professionnel et horodaté.

- **mode_maintenance.py**

C'est un script d'intervention manuel destiné à l'administrateur. Lors d'une mise à jour légitime du site Web (qui modifierait inévitablement le Hash et déclencherait les défenses), ce script permet de suspendre temporairement les services drac et drac-webhook. Une fois les modifications du site effectuées par le webmaster, le script recalcule automatiquement la **nouvelle empreinte SHA-256**, met à jour le fichier .env, ordonne à Proxmox de créer un nouveau Snapshot de référence sain, puis réarme les boucliers de sécurité.

2. Les Orchestrateurs

- **main.py**

Ce script est le cœur du service **drac.service**. Il tourne en boucle infinie (toutes les 15 secondes) et importe les modules périphériques décrits ci-dessus. Il orchestre la surveillance proactive : il appelle **detector.py**, analyse les résultats, et si une défaillance ou une modification de l'empreinte SHA-256 est avérée, il fait appel à **reactor.py** pour réparer la machine et à **notifier.py** pour alerter l'Administrateur.

- **webhook_drac.py**

Ce script est le cœur du service **drac-webhook.service**. Contrairement à **main.py** qui "cherche" les failles, ce module "écoute" passivement en exposant une API web grâce au framework Flask (sur le port 5000). Il possède deux routes principales :

- **/wazuh-alert**

Il réceptionne les alertes JSON du système HIDS (Wazuh), analyse le niveau de l'alerte et la description de la règle (ex: Règle Sudo, Force Brute) pour déclencher le bon Playbook dans **reactor.py**.

- **/slips-alert**

Il reçoit les alertes JSON du système NIDS (SLIPS), extrait l'adresse IP de l'attaquant détectée par l'Intelligence Artificielle et ordonne son bannissement immédiat.